

GPUMODE Lecture Study Notes: Lecture 105: cuDNN MXFP8 Attention

Core Problem the Lecture Addresses

This lecture explains how the NVIDIA cuDNN team designs high-performance scaled dot-product attention kernels for modern GPUs, with a special focus on FP8 and MXFP8 attention. The central engineering problem is:

How can an attention kernel exploit the higher tensor-core throughput of low-precision formats while preserving numerical quality and avoiding new bottlenecks in softmax, memory movement, shared memory, tensor memory, and synchronization?

The lecture begins from BF16 and FP16 scaled dot-product attention, then progressively transforms the pipeline into FP8 and MXFP8. The important point is that lower precision does not automatically imply faster attention. Attention is not only matrix multiplication. It also includes softmax, exponential operations, row reductions, intermediate quantization, scale handling, memory staging, and synchronization.

The main performance tension is:

FP8 or MXFP8 increases MMA throughput,

but

softmax, dequantization, quantization, scale loading, and synchronization do not automatically shrink.

Therefore, the kernel designer must hide or reduce the non-MMA work so that the increased tensor-core throughput is actually visible in end-to-end performance.

The lecture uses cuDNN attention as the concrete target. cuDNN is a deeply optimized library for deep learning operations and is used underneath frameworks such as PyTorch, JAX, Megatron-LM, Transformer Engine, vLLM, and FlashInfer. The cuDNN team has also been open sourcing kernels through the cuDNN

frontend repository, including kernels for attention, norms, Mixture-of-Experts style operations, and sparse or native attention variants.

The specific kernel family discussed is persistent scaled dot-product attention on GB300-like hardware. The lecture focuses on how to reason about:

- tile shapes such as $128 \times 128 \times 128$,
- one query tile versus two query tiles,
- one-CTA MMA versus two-CTA MMA,
- shared-memory buffer depth,
- tensor-memory pressure,
- FP8 intermediate quantization,
- MXFP8 block scaling,
- deterministic versus atomic gradient accumulation,
- forward and backward pipelines.

Required Background

Scaled Dot-Product Attention

For query matrix Q , key matrix K , and value matrix V , scaled dot-product attention computes:

$$S = \alpha QK^T,$$

where α is usually:

$$\alpha = \frac{1}{\sqrt{d}},$$

and d is the head dimension.

The attention probabilities are:

$$P = \text{softmax}(S),$$

and the output is:

$$O = PV.$$

For one attention head, common dimensions are:

$$Q \in \mathbb{R}^{M \times d}, \quad K \in \mathbb{R}^{N \times d}, \quad V \in \mathbb{R}^{N \times d_v}, \quad O \in \mathbb{R}^{M \times d_v}.$$

Here:

- M is the query sequence length or query tile extent,
- N is the key-value sequence length or key-value tile extent,
- d is the query-key head dimension,
- d_v is the value head dimension.

In many transformer models, $d = d_v$, but modern architectures may use variants such as $d = 192$ for query-key projection and $d_v = 128$ for value projection.

FlashAttention-Style Tiling

The key idea behind FlashAttention-style attention is to avoid materializing the full score matrix S and probability matrix P in global memory. Instead, the kernel streams key-value tiles through shared memory and maintains online softmax statistics.

A query tile attends over multiple key-value tiles. For a query tile Q_i and key-value tile K_j, V_j , the kernel computes:

$$S_{ij} = \alpha Q_i K_j^T.$$

It then updates row-wise softmax statistics and accumulates:

$$O_i \leftarrow O_i + P_{ij} V_j.$$

The online softmax update maintains, for each row, a running maximum and running denominator. If tile j produces scores S_{ij} , then:

$$m_{ij} = \max_n S_{ij,n},$$

and the global running row maximum is updated as:

$$m_i^{\text{new}} = \max(m_i^{\text{old}}, m_{ij}).$$

The softmax denominator is updated by rescaling the old denominator:

$$\ell_i^{\text{new}} = \exp(m_i^{\text{old}} - m_i^{\text{new}}) \ell_i^{\text{old}} + \sum_n \exp(S_{ij,n} - m_i^{\text{new}}).$$

The output accumulator is similarly rescaled:

$$O_i^{\text{new}} = \exp(m_i^{\text{old}} - m_i^{\text{new}}) O_i^{\text{old}} + P_{ij} V_j.$$

This is why attention kernels are not merely GEMM kernels. They combine tensor-core matrix multiplication with row-wise reductions, exponentials, normalization, and carefully ordered accumulation.

GPU Execution Concepts

The lecture assumes familiarity with the NVIDIA GPU execution model.

Streaming Multiprocessor

A streaming multiprocessor, abbreviated SM, is the main execution unit of the GPU. It contains warp schedulers, tensor cores, CUDA cores, registers, shared memory, and access to L1 and L2 cache.

Warp

A warp is a group of 32 threads that execute together. Tensor-core instructions are issued by warps or warp groups depending on the architecture and instruction family.

CTA

A CTA, or cooperative thread array, is CUDA terminology for a thread block. In this lecture, the kernels are persistent and often designed around one CTA per SM.

Shared Memory

Shared memory is an explicitly managed on-chip memory region used to stage tiles of Q , K , V , scales, or other intermediate data. It is much faster than global memory but limited in capacity and bandwidth.

Tensor Memory

The lecture refers to tensor memory, abbreviated TMEM or TMM in the transcript. This is a hardware resource associated with tensor-core operations and accumulators on newer architectures. The exact architectural details are hardware-specific, but for pipeline design the key point is that tensor memory is a limited on-chip resource that must hold MMA accumulators and, for MXFP8, sometimes scale factors.

MMA

MMA stands for matrix multiply-accumulate. Tensor cores execute instructions of the form:

$$D \leftarrow AB + C.$$

For attention, the two main forward MMAs are:

$$S = QK^T,$$

and:

$$O = PV.$$

In backward propagation, more MMAs appear, including computations for gradients with respect to Q , K , and V .

Main Concepts

cuDNN Attention as a Backend

cuDNN provides optimized implementations of deep learning operations across architectures, CUDA versions, data types, and operation variants. Attention support is exposed through several paths:

- coarse-grained APIs used by frameworks such as PyTorch and JAX,
- finer-grained Python APIs for cuDNN attention features,
- Transformer Engine APIs that call into cuDNN backends,
- inference stacks such as vLLM and FlashInfer using cuDNN as a backend.

The finer-grained cuDNN attention APIs expose support for:

- FP16 and BF16 attention,
- FP8 attention,
- MXFP8 attention,
- variable sequence length,
- ragged offsets,
- packed sequences,
- attention sinks,
- custom masks and graph-like attention patterns.

The lecture emphasizes that cuDNN is not just a black-box vendor library. The cuDNN frontend repository is becoming a place where kernel implementations can be inspected, adapted, and extended by external contributors.

Persistent Attention Kernels

The pipelines discussed are persistent kernels. A persistent kernel keeps CTAs resident on SMs and repeatedly processes tiles instead of launching many small independent CTAs.

For the lecture’s baseline model:

$$\text{one CTA} \longleftrightarrow \text{one SM.}$$

The benefit is that the kernel can maintain long-lived state in registers, shared memory, and tensor memory, while streaming key-value tiles through the SM.

Persistent attention is useful because attention has repeated structure:

$$Q_i \text{ attends over } K_0, K_1, \dots, K_{T-1} \text{ and } V_0, V_1, \dots, V_{T-1}.$$

A persistent CTA can keep a query tile resident and stream many key-value tiles through it.

GB300-Oriented Resource Assumptions

The lecture uses GB300-like hardware as the reference point. The important resource assumptions are:

$$\text{TMEM capacity} \approx 256 \text{ KB,}$$

and:

$$\text{shared-memory capacity} \approx 228 \text{ KB.}$$

The exact numbers are architecture-dependent, but the method generalizes. A pipeline should be designed by asking:

- How many MMA accumulators must be live?
- How many Q , K , and V tiles must be staged?
- How many buffers are needed to hide global-memory latency?
- How much register capacity does softmax require?
- Does the pipeline spill registers?

- Does the pipeline exceed shared-memory capacity?
- Does MXFP8 require scale factors to occupy tensor memory?

Forward Attention Building Blocks

The forward pass consists of three major components:

$$QK^T \rightarrow \text{softmax} \rightarrow PV.$$

The lecture models the forward pipeline using relative time bars. A useful abstraction is:

$$T_{\text{fwd}} = T_{QK} + T_{\text{softmax}} + T_{PV},$$

where:

$$T_{\text{softmax}} = T_{\text{max}} + T_{\text{exp}} + T_{\text{rowsum}}.$$

On the GB300-like target, the row-wise maximum can be implemented efficiently enough that the lecture says it does not dominate the pipeline. The exponential operation is the critical CUDA-math component.

Baseline Tile Shape

A common baseline tile is:

$$M_{\text{tile}} = 128, \quad N_{\text{tile}} = 128, \quad K_{\text{tile}} = 128.$$

This corresponds to a tile work unit:

$$128 \times 128 \times 128.$$

The dimensions mean:

- M_{tile} : number of query rows in the query tile,
- N_{tile} : number of key-value tokens in the streamed tile,
- K_{tile} : head dimension or contracting dimension.

For Llama-like attention, the head dimension is typically:

$$d = 128.$$

For DeepSeek V3 style MLA shapes, the query-key contracting dimension can be:

$$d = 192.$$

This increases the relative cost of the QK^T MMA compared with the PV MMA.

Why Tile Width Matters for Softmax

The softmax is performed row-wise across the key dimension. If N_{tile} is larger, each row has more elements to reduce and exponentiate.

A larger N_{tile} can improve arithmetic granularity and amortization, but it also increases register pressure because the softmax intermediate values live in registers.

A simplified register pressure model is:

$$R_{\text{softmax}} \propto M_{\text{tile}} \cdot N_{\text{tile}} \cdot r_{\text{per element}},$$

where $r_{\text{per element}}$ represents the amount of register state needed per score or probability value.

If N_{tile} becomes too large, the kernel may spill registers to local memory:

$$R_{\text{needed}} > R_{\text{available}} \Rightarrow \text{register spilling.}$$

Register spilling is very expensive because local memory is backed by global memory or cache hierarchy, not by true registers.

One Query Tile Pipeline

The simplest forward pipeline holds one query tile and streams key-value tiles. A single iteration looks like:

$$QK^T \rightarrow \text{softmax} \rightarrow PV.$$

For key-value chunks indexed by j , the loop is:

$$\text{for } j = 0, \dots, T - 1 : \quad S_j = QK_j^T, \quad P_j = \text{softmax}(S_j), \quad O += P_j V_j.$$

The weakness of this naive pipeline is that tensor cores are not issuing MMA instructions continuously. There are gaps where the pipeline performs softmax math instead of MMA.

If the tensor-core utilization is U_{TC} , then:

$$U_{\text{TC}} = \frac{T_{\text{MMA}}}{T_{\text{MMA}} + T_{\text{non-MMA gaps}}}.$$

The goal is to make:

$$T_{\text{non-MMA gaps}} \approx 0,$$

so that:

$$U_{\text{TC}} \approx 1.$$

Two Query Tile Pipeline

A common solution is to process two query tiles in an interleaved fashion. Instead of one query tile waiting while softmax executes, the kernel overlaps the MMA of one query tile with the softmax of another query tile.

Conceptually:

$$\begin{aligned} Q_0 K^T &\rightarrow \text{softmax}(Q_0 K^T) \rightarrow P_0 V, \\ Q_1 K^T &\rightarrow \text{softmax}(Q_1 K^T) \rightarrow P_1 V. \end{aligned}$$

The schedule interleaves these so that tensor cores can keep issuing MMA instructions.

The benefit is:

$$U_{\text{TC}} \rightarrow 100\%.$$

The cost is increased on-chip resource usage:

two query tiles $\Rightarrow$ more accumulators, more live state, more shared-memory pressure.

Global-Memory Latency and Buffer Depth

Even if tensor-core utilization appears high in a diagram, the kernel may stall if K and V tiles are not loaded into shared memory in time.

Let:

$$T_{\text{GMEM}}$$

be the global-memory latency for loading a key or value tile, and let:

$$T_{\text{cover}}$$

be the amount of independent computation available while that tile is being loaded. To hide memory latency:

$$T_{\text{cover}} \geq T_{\text{GMEM}}.$$

The lecture states that for DeepSeek-like shapes, empirically hiding global-memory latency behind roughly four to five MMAs is effective:

$$T_{\text{GMEM}} \approx 4T_{\text{MMA}} \text{ to } 5T_{\text{MMA}}.$$

Therefore, one should use as many key-value buffers as possible, subject to shared-memory capacity.

If each K or V tile consumes B_{tile} bytes and there are s stages, the shared-memory requirement is approximately:

$$B_{\text{SMEM}} \approx s(B_K + B_V) + B_Q + B_{\text{other}}.$$

The constraint is:

$$B_{\text{SMEM}} \leq B_{\text{SMEM},\text{max}}.$$

Two-CTA MMA

For larger head dimensions such as $d = 192$, each key tile becomes larger. For example, if the tile shape is:

$$N_{\text{tile}} = 128, \quad d = 192,$$

then the key tile has:

$$128 \cdot 192 = 24576$$

elements.

For BF16, each element is 2 bytes, so the tile size is:

$$24576 \cdot 2 = 49152 \text{ bytes} = 48 \text{ KB}.$$

This can exceed the shared-memory budget when multiple buffers are used.

The lecture describes using two-CTA MMA to reduce shared-memory consumption for operand B . In simplified form:

$$\text{two-CTA MMA} \Rightarrow B_{\text{operand } B, \text{ per CTA}} \approx \frac{1}{2} B_{\text{operand } B}.$$

This makes room for more K and V buffers.

However, two-CTA MMA is not universally better. It increases effective tile granularity. The lecture gives the intuition:

$$128 \times 128 \rightarrow 256 \times 256 \rightarrow 512 \times 512$$

as one stacks two query tiles and two-CTA MMA effects. Larger effective tiles may increase quantization effects, masking cost, and wasted work for causal or irregular masks.

There Is No Universal Pipeline

The lecture repeatedly emphasizes:

There is no single attention pipeline that is optimal for every model, head dimension, mask, sequence length, precision, and architecture.

Examples:

- Llama-like $d = 128$ forward attention may prefer two query tiles.
- DeepSeek-like $d = 192$ may need two-CTA MMA to fit more buffers.
- Very large head dimensions, such as Qwen 3.5 style shapes, may prefer one query tile.
- Non-causal attention may benefit from two-CTA MMA.
- Causal masks may prefer one-CTA MMA because large effective tiles waste work.
- Flexible masks may require smaller tiles to fit extra mask math in registers.
- Low-precision attention may prefer different MMA overlap because softmax does not shrink with MMA throughput.

The correct design method is to build a resource and latency model, then tune.

Hardware-Level Explanation

Pipeline Resources

The lecture models attention using several hardware resources:

- tensor cores for MMA,
- CUDA cores or special-function units for exponentials and FP32 operations,
- global memory for input and output tensors,
- L2 cache and swizzled access patterns for reducing memory latency,
- shared memory for staged Q , K , V , gradients, and scales,
- tensor memory for accumulators and MXFP8 scale factors,
- registers for softmax state and per-thread temporary values.

An attention kernel is fast only when the schedule balances all these resources.

MMA Utilization

For a pipeline with total runtime:

$$T_{\text{total}} = T_{\text{MMA}} + T_{\text{softmax exposed}} + T_{\text{memory exposed}} + T_{\text{sync exposed}},$$

the effective tensor-core utilization is:

$$U_{\text{TC}} = \frac{T_{\text{MMA}}}{T_{\text{total}}}.$$

The goal is:

$$T_{\text{softmax exposed}} \approx T_{\text{memory exposed}} \approx T_{\text{sync exposed}} \approx 0.$$

This does not mean those operations disappear. It means they are overlapped with MMA or moved outside the critical path.

Shared-Memory Capacity

Shared memory stores tiles and sometimes gradient accumulation buffers. The basic tile size for a matrix A with shape $m \times n$ and element size b bytes is:

$$B(A) = mnb.$$

For a BF16 key tile with $N_{\text{tile}} = 128$ and $d = 128$:

$$B(K) = 128 \cdot 128 \cdot 2 = 32768 \text{ bytes} = 32 \text{ KB}.$$

For $d = 192$:

$$B(K) = 128 \cdot 192 \cdot 2 = 49152 \text{ bytes} = 48 \text{ KB}.$$

If the kernel wants four K buffers and four V buffers, the rough memory requirement can grow quickly:

$$B_{KV} = s(B(K) + B(V)).$$

For BF16 Llama-style $d = 128$, assuming $B(K) = B(V) = 32 \text{ KB}$ and $s = 3$, one already needs:

$$B_{KV} = 3(32 + 32) \text{ KB} = 192 \text{ KB}.$$

This leaves little room for Q , scales, synchronization state, or other buffers under a 228 KB shared-memory budget.

Tensor-Memory Capacity

Tensor memory holds MMA accumulators and, for MXFP8, scale factors. If an accumulator tile has shape $M_{\text{tile}} \times N_{\text{tile}}$ and uses FP32 accumulation, then the accumulator storage is:

$$B_{\text{acc}} = M_{\text{tile}} N_{\text{tile}} \cdot 4.$$

For 128×128 :

$$B_{\text{acc}} = 128 \cdot 128 \cdot 4 = 65536 \text{ bytes} = 64 \text{ KB}.$$

Multiple live accumulators can quickly consume tensor memory:

$$B_{\text{TMEM}} = \sum_i B_{\text{acc},i} + B_{\text{scales}} + B_{\text{other}}.$$

The constraint is:

$$B_{\text{TMEM}} \leq 256 \text{ KB}.$$

MXFP8 makes this harder because scale factors must be copied into tensor memory before the MMA.

Memory Coalescing and Swizzling

The lecture mentions L2 swizzling, a technique also popular in FlashAttention-style kernels. The idea is to schedule CTAs and memory accesses so that adjacent or related CTAs reuse data through L2 cache instead of repeatedly loading the same data from HBM.

The performance goal is to reduce effective global-memory latency:

$$T_{\text{GMEM, effective}} < T_{\text{GMEM, raw}}.$$

If L2 reuse improves, the number of MMA operations needed to hide memory latency may decrease.

Softmax Hardware Behavior

Softmax uses operations that are not tensor-core MMAs:

$$\max, \quad \exp, \quad \sum, \quad \text{normalization}.$$

The expensive operation is the exponential. In many kernels, the softmax uses base-2 exponentials because they map well to GPU hardware and can absorb scale factors conveniently:

$$\exp(x) = 2^{x \log_2 e}.$$

If the attention scale is α , then the score entering base-2 exponential can be written as:

$$x = \alpha \log_2(e) \cdot QK^T.$$

This is why the lecture writes softmax logic using $\exp_2$.

Mathematical Reasoning

Forward Attention

The forward attention equations are:

$$S = \alpha QK^T,$$

$$P = \text{softmax}(S),$$

$$O = PV.$$

For numerical stability:

$$P_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_k \exp(S_{ik} - m_i)},$$

where:

$$m_i = \max_j S_{ij}.$$

Using base-2 exponentials:

$$P_{ij} = \frac{2^{(S_{ij} - m_i) \log_2 e}}{\sum_k 2^{(S_{ik} - m_i) \log_2 e}}.$$

Relative Work Model for Forward Pass

Let:

$$T_{QK}$$

be the time for the first MMA,

$$T_{\text{exp}}$$

be the dominant softmax exponential time, and:

$$T_{PV}$$

be the second MMA time.

For BF16 with equal head dimensions:

$$T_{QK} \approx T_{PV}.$$

For DeepSeek-style MLA with $d_{QK} = 192$ and $d_V = 128$:

$$T_{QK} \approx \frac{192}{128} T_{PV} = 1.5 T_{PV}.$$

This changes the schedule because the first MMA now creates more opportunity to hide softmax or memory work, but it also increases tile storage.

Backward Attention

Let dO be the gradient of the output. The backward pass computes gradients dQ , dK , and dV .

The standard relationships are:

$$dV = P^T dO,$$

$$dP = dOV^T,$$

and for softmax:

$$dS_{ij} = P_{ij} \left(dP_{ij} - \sum_k P_{ik} dP_{ik} \right).$$

Then:

$$dQ = dSK,$$

$$dK = dS^T Q.$$

The backward pass therefore contains several matrix multiplications:

$$dV = P^T dO,$$

$$dP = dOV^T,$$

$$dS = \text{dsoftmax}(P, dP),$$

$$dQ = dSK,$$

$$dK = dS^T Q.$$

The lecture counts five main BMMs in the one-kernel backward formulation and describes a two-kernel version that repeats some softmax-related work, increasing the number of BMMs from five to seven.

Two-Kernel Backward Decomposition

A two-kernel backward approach separates gradient computation into:

DKV kernel and DQ kernel.

The DKV kernel computes:

$$dV = P^T dO,$$

and:

$$dK = dS^T Q.$$

The DQ kernel computes:

$$dQ = dSK.$$

The trade-off is:

two-kernel backward $\Rightarrow$ more MMA and repeated CUDA math,

but:

two-kernel backward $\Rightarrow$ deterministic gradients and less atomic pressure.

Atomic Accumulation Issue

If the backward kernel streams over key-value tiles, then multiple CTAs or loop iterations may contribute to the same dQ . This requires atomic accumulation:

$$dQ_i \leftarrow dQ_i + \Delta dQ_{i,j}.$$

Atomic floating-point addition is not deterministic in general because addition order may vary:

$$(a + b) + c \neq a + (b + c)$$

under finite-precision arithmetic.

Therefore, one-kernel backward may introduce nondeterminism unless extra scheduling and synchronization mechanisms are used.

The lecture mentions semaphores and shortest-processing-time scheduling as mitigation strategies used in FlashAttention-style approaches, but these mechanisms have overhead.

FP8 Quantization

Classical FP8 uses a per-tensor scale. A real tensor value x is represented by an FP8 value q and scale s approximately as:

$$x \approx \frac{q}{s}.$$

Equivalently, before storing in FP8:

$$q = \text{quantize}_{\text{FP8}}(sx).$$

The purpose of the scale is to map the dynamic range of x into the representable range of the FP8 format.

If the maximum finite FP8 value is $x_{\max}^{\text{FP8}}$ and the tensor amax is:

$$a = \max_i |x_i|,$$

a naive scale would be:

$$s_{\text{naive}} = \frac{x_{\max}^{\text{FP8}}}{a}.$$

For E4M3, the lecture uses:

$$x_{\max}^{\text{E4M3}} = 448.$$

If $a = 1$, the naive scale is:

$$s_{\text{naive}} = 448.$$

Power-of-Two Scaling

The lecture explains that the best practical scale is not necessarily:

$$\frac{x_{\max}^{\text{FP8}}}{a}.$$

Instead, the recipe is to choose the largest power of two less than or equal to that ratio:

$$s = 2^{\left\lfloor \log_2 \left(\frac{x_{\max}^{\text{FPS}}}{a} \right) \right\rfloor}.$$

For $x_{\max}^{\text{E4M3}} = 448$ and $a = 1$:

$$s = 2^{\lfloor \log_2(448) \rfloor} = 2^8 = 256.$$

The reason is that multiplying by a power of two shifts only the exponent field of a floating-point value. It does not disturb the mantissa.

Floating-point numbers have the rough form:

$$x = (-1)^s \cdot 2^{e-b} \cdot (1.m),$$

where:

- s is the sign bit,
- e is the encoded exponent,
- b is the exponent bias,
- m is the mantissa.

Multiplication by 2^k gives:

$$2^k x = (-1)^s \cdot 2^{e-b+k} \cdot (1.m).$$

Thus the mantissa remains unchanged. This preserves representability for normal values and avoids introducing extra rounding error that would occur with arbitrary non-power-of-two scales.

Intermediate Tensor Quantization

The user provides scales for input tensors such as Q , K , V , and dO . However, the kernel internally creates intermediate tensors:

$$P = \text{softmax}(S),$$

and in backward:

$$dS = \text{dsoftmax}(P, dP).$$

These tensors are internal to the kernel. The user cannot directly provide their scales. Therefore, the kernel must choose quantization scales for P and dS .

Scale Fusion for FP8 Forward

Suppose Q and K are stored in FP8 with scales s_Q and s_K . The dequantized values are approximately:

$$Q_{\text{real}} \approx \frac{Q_{\text{fp8}}}{s_Q}, \quad K_{\text{real}} \approx \frac{K_{\text{fp8}}}{s_K}.$$

Then:

$$S = \alpha Q_{\text{real}} K_{\text{real}}^T \approx \alpha \frac{Q_{\text{fp8}} K_{\text{fp8}}^T}{s_Q s_K}.$$

For base-2 softmax, the exponent input includes:

$$\gamma = \frac{\alpha \log_2 e}{s_Q s_K}.$$

Thus:

$$P_{ij} = 2^{\gamma(Q_{\text{fp8}} K_{\text{fp8}}^T)_{ij} - \text{LSE}_i},$$

up to the row-wise normalization convention.

Because s_Q , s_K , and α are per-tensor constants, γ can be computed once outside the main loop. This has almost no inner-loop overhead.

Quantizing the Softmax Probability Tensor

The PV MMA requires P and V to be in compatible low-precision formats. Therefore P , which is naturally FP32 after softmax, must be quantized to FP8 or BF16 before the PV MMA.

A naive formula would be:

$$P_{\text{scaled}} = s_P \cdot 2^x,$$

where x is the softmax exponent input. This would require an extra FP32 multiply for every element:

$$T_{\text{softmax}} \leftarrow T_{\text{softmax}} + T_{\text{elementwise scale}}.$$

The lecture presents a better trick:

$$s_P 2^x = 2^{x + \log_2(s_P)}.$$

Thus the scale can be moved inside the exponent as an additive term:

$$P_{\text{scaled}} = 2^{x + \log_2(s_P)}.$$

Since the log-scale adjustment is row-wise and can be folded into the log-sum-exp quantity, it avoids a full 128×128 elementwise multiplication. Instead of:

$$M_{\text{tile}} N_{\text{tile}}$$

extra multiplications, the kernel pays roughly:

$$M_{\text{tile}}$$

row-wise adjustments.

For 128×128 , this changes the extra work from:

$$16384 \text{ operations}$$

to roughly:

$$128 \text{ operations.}$$

Choosing the Scale for P

Softmax outputs satisfy:

$$0 \leq P_{ij} \leq 1.$$

Therefore:

$$\max_{i,j} |P_{ij}| \leq 1.$$

The kernel can use:

$$a_P \approx 1.$$

Then the power-of-two scale is:

$$s_P = 2^{\lfloor \log_2(x_{\text{max}}^{\text{FP8}}) \rfloor}.$$

For E4M3:

$$s_P = 2^{\lfloor \log_2(448) \rfloor} = 256.$$

This is conservative because the true amax of P may be less than 1, but it is simple, stable, and works well.

Quantizing dS

Unlike P , the tensor dS is not bounded by softmax output range. It is computed from products and sums involving P , dP , dO , and V . A simplified formula is:

$$dS_{ij} = P_{ij} \left(dP_{ij} - \sum_k P_{ik} dP_{ik} \right).$$

The value of dS can vary widely:

$$dS_{ij} \in \mathbb{R}.$$

Therefore, no simple bound like $a_{dS} = 1$ exists.

For classical FP8, the lecture describes using a historical amax:

$$a_{dS}^{(t)} \approx a_{dS}^{(t-1)}.$$

Then:

$$s_{dS}^{(t)} = 2^{\left\lfloor \log_2 \left(\frac{x_{\text{FP8}}^{\text{max}}}{a_{dS}^{(t-1)}} \right) \right\rfloor}.$$

The kernel can also output the current amax:

$$a_{dS}^{(t)} = \max_{i,j} |dS_{ij}^{(t)}|,$$

for use in a later iteration.

The reason not to use the current amax immediately is that a per-tensor current amax would require computing all of dS , reducing it globally, then rerunning or continuing the MMA. That would stall the pipeline.

MXFP8 Block Scaling

MXFP8 replaces one per-tensor scale with per-block scales. Instead of:

$$s_X \quad \text{for the entire tensor,}$$

we have:

$$s_{X,b} \quad \text{for block } b.$$

For a block B_b , the amax is:

$$a_b = \max_{i \in B_b} |x_i|.$$

The block scale is:

$$s_b = 2^{\left\lfloor \log_2 \left(\frac{x_{\max}^{\text{FP8}}}{a_b} \right) \right\rfloor}.$$

This improves precision because each block adapts to its own dynamic range.

The trade-off is extra overhead:

more scales $\Rightarrow$ more loads, copies, tensor-memory pressure, and synchronization.

One-Dimensional versus Two-Dimensional MXFP8 Scaling

The lecture compares 1D block scaling and 2D block scaling.

In 1D block scaling, scales are defined along one dimension. For example, blocks of 32 elements along the contracting dimension may share a scale.

In 2D block scaling, a scale covers a two-dimensional block:

$$B_{p,q} = \{(i, j) : i \in I_p, j \in J_q\}.$$

The lecture states that 2D scaling is less expressive because more elements share the same scale. Also, native hardware support is oriented toward 1D scaling. Therefore, the lecture focuses on 1D MXFP8.

Contracting Dimension Constraint

MXFP8 MMA hardware requires scales to align with the contracting dimension of the matrix multiplication.

For an MMA:

$$D = AB,$$

where:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N},$$

the contracting dimension is K .

The scale layout must correspond to this K dimension. In attention backward, the same tensor may participate in different MMAs with different contracting dimensions. Therefore, the kernel may need both row-wise and column-wise scale formats for the same logical tensor.

For example, Q may be used in one MMA where the head dimension is contracting and another where the sequence dimension is contracting. Therefore, scales may be needed in both orientations.

Code Walkthrough

Representative Forward Attention Loop

The lecture does not provide a full CUDA source listing, but the pipeline can be represented with the following pseudocode. This is not production code; it shows the conceptual dataflow.

```
for each persistent_CTA_on_SM {
    load Q_tile into registers_or_shared_memory;

    initialize row_max = -infinity;
    initialize row_sum = 0;
    initialize O_accumulator = 0;

    for kv_tile_id in range(num_kv_tiles) {
        async_load K_tile[kv_stage] from global_memory to shared_memory;
        async_load V_tile[kv_stage] from global_memory to shared_memory;

        wait_until K_tile and V_tile are ready;

        S_accumulator = mma(Q_tile, transpose(K_tile));
```



```

        update row_max using S_accumulator;

        P_tile = exp2(S_accumulator - row_logsumexp);

        row_sum = update_softmax_denominator(P_tile);

        P_quantized = quantize_intermediate_probability(P_tile);

        O_accumulator += mma(P_quantized, V_tile);
    }

    normalize O_accumulator using row_sum;
    store O_tile to global_memory;
}

```

The important hardware behaviors are:

- Q is kept resident while K and V stream through.
- K and V are staged through shared memory.
- QK^T and PV are tensor-core MMAs.
- Softmax uses CUDA math and row-wise reductions.
- The intermediate probability tensor P must be quantized before the PV MMA when using FP8 or MXFP8.
- The loop must be pipelined so that loads and softmax do not expose bubbles.

Representative Two-Query-Tile Schedule

A two-query-tile pipeline can be described as:

```

for each persistent_CTA_on_SM {
    load Q0_tile;
    load Q1_tile;

    prefetch K_tile[0], V_tile[0];

    for kv_tile_id in range(num_kv_tiles) {
        prefetch K_tile[next], V_tile[next];

        S0 = mma(Q0_tile, transpose(K_tile[current]));
        S1 = mma(Q1_tile, transpose(K_tile[current]));

        P0 = softmax_fragment(S0);
        O0 += mma(quantize(P0), V_tile[current]);
    }
}

```

```

        P1 = softmax_fragment(S1);
        O1 += mma(quantize(P1), V_tile[current]);
    }

    store O0;
    store O1;
}

```

The key idea is not that the code literally executes in this sequential order. A real kernel overlaps operations across warp groups and pipeline stages. The pseudocode shows the dependency structure.

The two-query-tile schedule increases the amount of independent MMA work available:

$$T_{\text{cover}} \approx T_{Q_0K} + T_{Q_1K} + T_{P_0V} + T_{P_1V}.$$

This makes it easier to cover:

$$T_{\text{exp}} + T_{\text{GMEM}}.$$

FP8 Scale Folding in Softmax

The scale folding trick can be represented as:

```

float qk_scale = attention_scale * log2e / (scale_q * scale_k);
float log2_scale_p = log2f(scale_p);

for each score_element {
    float x = qk_scale * qk_accumulator_element - row_lse;
    float p_scaled = exp2f(x + log2_scale_p);
    fp8 p_fp8 = quantize_to_fp8(p_scaled);
}

```

A naive version would do:

```

float p = exp2f(x);
float p_scaled = scale_p * p;
fp8 p_fp8 = quantize_to_fp8(p_scaled);

```

The optimized form moves the multiplication into the exponent:

$$s_P 2^x = 2^{x + \log_2(s_P)}.$$

This removes an elementwise FP32 multiplication from the critical path.

Power-of-Two Scale Selection

A representative scale-selection function is:

```
float choose_power_of_two_scale(float fp8_max, float amax) {  
    float ratio = fp8_max / amax;  
    float exponent = floorf(log2f(ratio));  
    return exp2f(exponent);  
}
```

This implements:

$$s = 2^{\lfloor \log_2(x_{\max}^{\text{FP8}}/a) \rfloor}.$$

The function should handle edge cases in production code, such as:

$$a = 0, \quad a = \text{NaN}, \quad a = \infty.$$

A robust implementation might clamp the scale to a supported range.

MXFP8 Online Quantization of dS

For MXFP8, dS cannot simply use a previous per-tensor amax if one wants true block-level scaling. The kernel must compute block amax values online.

```
for each block_of_32_values in dS_fragment {  
    float amax = 0.0f;  
  
    for i in 0..31 {  
        amax = max(amax, abs(dS_fragment[i]));  
    }  
  
    float scale = choose_power_of_two_scale(fp8_max, amax);  
  
    for i in 0..31 {  
        dS_fp8[i] = quantize_to_fp8(scale * dS_fragment[i]);  
    }  
  
    store_or_stage_scale(scale);  
}
```

This adds work:

$$T_{\text{MXFP8 quant}} = T_{\text{amax}} + T_{\text{scale}} + T_{\text{elementwisemultiply}} + T_{\text{quantize}}.$$

Unlike classical FP8, this cannot be fully fused away because the scale is not known until the block values are computed.

MXFP8 Scale Copy into Tensor Memory

The lecture describes an additional hardware requirement: MXFP8 MMA scale factors must reside in tensor memory. The data path is:

global memory $\rightarrow$ shared memory $\rightarrow$ tensor memory $\rightarrow$ MMA.

A conceptual pseudocode representation is:

```
tma_load(data_tile, global_memory, shared_memory);
tma_load(scale_tile, global_memory, shared_memory);

wait_until_shared_memory_ready();

tcgen05_copy_scales(shared_memory_scale_tile, tensor_memory_scale_tile);

mma_mxfp8(
    data_operand_a,
    data_operand_b,
    tensor_memory_scale_a,
    tensor_memory_scale_b,
    accumulator
);
```

The lecture notes that hardware provides an interlock between the scale-copy instruction and the MMA instruction, so no extra explicit barrier is needed between those two instructions if they are issued in the correct order.

However, the copy itself consumes cycles, and the scales consume tensor-memory capacity. Therefore MXFP8 has a performance penalty relative to plain FP8.

Performance Intuition

Why FP8 Can Be Faster

FP8 has higher MMA throughput than BF16 or FP16. The lecture describes the FP8 MMA throughput as effectively $2\times$ relative to BF16 for the relevant pipeline.

If:

$$T_{\text{MMA,BF16}} = T,$$

then roughly:

$$T_{\text{MMA,FP8}} \approx \frac{T}{2}.$$

This is beneficial only if non-MMA work does not dominate:

$$T_{\text{total,FP8}} = \frac{T_{\text{MMA,BF16}}}{2} + T_{\text{softmax}} + T_{\text{quantization}} + T_{\text{memory}} + T_{\text{sync}}.$$

If:

$$T_{\text{softmax}} \gg \frac{T_{\text{MMA,BF16}}}{2},$$

then the speedup will be much less than $2\times$.

Therefore, FP8 pipeline design must create more overlap for softmax and dsoftmax.

Forward FP8 Pipeline

For forward attention, the same basic two-query-tile BF16 pipeline often still works well for FP8. The reasons are:

- FP8 halves input tile storage compared with BF16.
- More key-value buffering fits in shared memory.
- P can be quantized efficiently using scale folding.
- Tensor-memory accumulator usage remains similar because accumulation is still FP32.

Shared-memory tile size for FP8 is half of BF16:

$$B_{\text{tile,FP8}} = \frac{1}{2} B_{\text{tile,BF16}}.$$

For a 128×128 tile:

$$B_{\text{tile,FP8}} = 128 \cdot 128 \cdot 1 = 16 \text{ KB}.$$

This makes deeper buffering easier.

Backward FP8 Pipeline

Backward attention is more complicated because dsoftmax and gradient accumulation remain expensive.

The lecture explains that when MMA time halves, the previous BF16 schedule may no longer cover the same softmax and dsoftmax work. The exposed gap increases because:

$$T_{\text{MMA}} \downarrow \quad \text{but} \quad T_{\text{dsoftmax}} \nmid \downarrow.$$

The solution is interloop overlap. The kernel overlaps independent MMAs from the next loop iteration with softmax or dsoftmax from the current iteration.

A simplified dependency relation is:

$$\text{MMA}_{j+1} \quad \text{can overlap with} \quad \text{dsoftmax}_j$$

if it does not consume results from dsoftmax_j .

For the DQ kernel, the lecture describes switching the order of two independent MMAs. This creates a larger window between producing S and needing dS , giving dsoftmax more time to complete without exposing a bubble.

Why Two-Kernel Backward Becomes More Attractive at Low Precision

FP8 reduces:

$$T_{\text{MMA}},$$

and reduces data movement for FP8 tensors, but it does not reduce all costs. In particular, dQ accumulation may still be FP32:

$$dQ_{\text{acc}} \in \text{FP32}.$$

Therefore, the relative cost of atomic dQ accumulation grows:

$$\frac{T_{\text{atomic FP32}}}{T_{\text{MMA,FP8}}} > \frac{T_{\text{atomic FP32}}}{T_{\text{MMA,BF16}}}.$$

This makes two-kernel backward more attractive because it avoids or reduces nondeterministic atomic accumulation.

Why MXFP8 Is Slower Than FP8

MXFP8 improves numerical accuracy but introduces several overheads:

- scale factors are per block, not per tensor,
- scale factors must be loaded with the data,
- scale factors must be copied from shared memory to tensor memory,
- tensor memory now stores both accumulators and scales,

- online quantization is needed for dS ,
- additional synchronization is required to avoid overwriting live scale buffers.

A simplified cost model is:

$$T_{\text{MXFP8}} = T_{\text{FP8 MMA}} + T_{\text{scale load}} + T_{\text{scale copy}} + T_{\text{online amax}} + T_{\text{scale apply}} + T_{\text{extra sync}}.$$

Thus:

$$T_{\text{MXFP8}} > T_{\text{FP8}},$$

but the accuracy is better.

Accuracy Advantage of MXFP8

The lecture compares FP8 and MXFP8 against an FP16-based reference using relative error metrics such as root mean square relative error.

A general RMS relative error can be written as:

$$\text{RMSRE}(X, \hat{X}) = \sqrt{\frac{1}{n} \sum_{i=1}^n \left(\frac{\hat{X}_i - X_i}{X_i + \epsilon} \right)^2}.$$

MXFP8 improves error because each block has its own scale:

$$s_{\text{global}} \rightarrow s_b.$$

If a tensor contains both small and large values, per-tensor scaling may waste representable range on the largest values. Per-block scaling adapts locally, preserving more information for small values in blocks where the local amax is smaller.

Common Misconceptions

Misconception: FP8 Automatically Gives a $2\times$ Faster Attention Kernel

FP8 doubles relevant MMA throughput, but attention includes softmax, dsoftmax, quantization, scale handling, and memory movement.

The true speedup is closer to:

$$\frac{T_{\text{BF16 MMA}} + T_{\text{other}}}{\frac{1}{2}T_{\text{BF16 MMA}} + T_{\text{other,FP8}}}.$$

If $T_{\text{other,FP8}}$ is large, the speedup is much less than $2\times$.

Misconception: One Pipeline Works for All Models

Tile shape and pipeline depth depend on:

- head dimension,
- sequence length,
- causal versus non-causal masking,
- flexible masks,
- precision,
- architecture,
- shared-memory capacity,
- tensor-memory capacity,
- register pressure.

A pipeline that works well for Llama-style $d = 128$ attention may not work well for DeepSeek-style $d = 192$ or Qwen-style larger head dimensions.

Misconception: Two-CTA MMA Is Always Better

Two-CTA MMA can reduce shared-memory pressure for operand B , but it also changes effective tile granularity and can increase wasted work for masks.

It is useful when:

$$B_{\text{SMEM needed}} > B_{\text{SMEM available}},$$

but harmful if it increases masking cost or tile quantization effects.

Misconception: MXFP8 Is Just FP8 with Better Scales

MXFP8 requires a different pipeline. It introduces block scale loading, scale copies into tensor memory, possible scale-buffer overlap with accumulators, online quantization for intermediate tensors, and additional synchronization hazards.

The numerical model improves, but the hardware pipeline becomes more complex.

Misconception: The Softmax Probability Tensor Does Not Need Quantization

In low-precision attention, the PV MMA requires both operands to be compatible with the MMA data type. Since V is FP8 or MXFP8, the intermediate P must also be quantized to the appropriate format before the MMA.

Misconception: The Best FP8 Scale Is Always $x_{\max}/a$

The lecture shows that arbitrary scaling can increase quantization error because it changes mantissa information. Power-of-two scaling is preferred:

$$s = 2^{\left\lfloor \log_2 \left(\frac{x_{\max}^{\text{FP8}}}{a} \right) \right\rfloor}.$$

This shifts exponent bits without perturbing the mantissa.

Practical Takeaways

Kernel Design Takeaways

The main engineering process is:

1. Break attention into MMA, softmax, dsoftmax, memory movement, and quantization stages.
2. Estimate relative time for each stage.
3. Choose tile sizes based on head dimension, mask type, and register pressure.
4. Use enough key-value buffers to hide global-memory latency.
5. Use two query tiles when it helps cover softmax and maintain tensor-core utilization.
6. Use two-CTA MMA when shared-memory capacity prevents enough buffering.
7. Reevaluate the pipeline when moving from BF16 to FP8 because MMA time shrinks.
8. Reevaluate again when moving to MXFP8 because scale handling becomes a first-class pipeline stage.

Numerical Takeaways

For classical FP8:

- use per-tensor scales for inputs,
- fuse Q , K , and attention scales into a single constant,

- use a known bound $a_P = 1$ for softmax probabilities,
- use historical amax for dS ,
- choose power-of-two scales.

For MXFP8:

- use block-level scales for better accuracy,
- compute online block amax for intermediate tensors such as dS ,
- respect hardware scale-layout requirements along the contracting dimension,
- account for scale movement into tensor memory.

Performance Takeaways

The important bottlenecks are:

- exposed softmax and dsoftmax cycles,
- global-memory latency for K and V ,
- shared-memory capacity for multi-stage buffering,
- shared-memory bandwidth in backward,
- FP32 atomic accumulation for gradients,
- tensor-memory pressure from accumulators and MXFP8 scales,
- synchronization needed for safe tensor-memory reuse.

A good attention kernel is not merely one with fast MMAs. It is one where all supporting operations are hidden or minimized.

Project or Experiment Ideas

Benchmark Naive Attention versus FlashAttention-Style Tiling

Implement a PyTorch baseline that materializes S and P , then compare it to a tiled implementation in Triton or CUDA. Measure memory traffic and runtime as sequence length increases.

Expected result:

$$\text{materialized attention memory} = O(N^2),$$

while tiled attention avoids writing the full $N \times N$ probability matrix to global memory.

Measure Tile Shape Effects

Implement a simple attention forward kernel with tile sizes:

$$64 \times 64, \quad 128 \times 64, \quad 128 \times 128, \quad 128 \times 256.$$

Measure:

- runtime,
- achieved occupancy,
- register spilling,
- shared-memory usage,
- tensor-core utilization.

Use Nsight Compute to inspect whether larger softmax tiles increase register pressure.

Compare BF16 and FP8 Softmax Exposure

Create a synthetic pipeline model where MMA time is halved but softmax time is fixed:

$$T_{\text{BF16}} = T_{\text{MMA}} + T_{\text{softmax}},$$

$$T_{\text{FP8}} = \frac{1}{2}T_{\text{MMA}} + T_{\text{softmax}}.$$

Plot speedup as a function of:

$$\frac{T_{\text{softmax}}}{T_{\text{MMA}}}.$$

This demonstrates why FP8 requires better overlap.

Implement Power-of-Two Scaling

Write a small Python experiment that quantizes random tensors using:

$$s_{\text{naive}} = \frac{x_{\text{max}}^{\text{FP8}}}{a},$$

and:

$$s_{\text{pow2}} = 2^{\left\lfloor \log_2 \left(\frac{x_{\text{max}}^{\text{FP8}}}{a} \right) \right\rfloor}.$$

Compare quantization error. The expected outcome is that power-of-two scaling avoids some cases where scaling increases error relative to no scaling.

Simulate Per-Tensor versus Per-Block Scaling

Generate tensors with heterogeneous magnitudes. For example, concatenate blocks with standard deviations:

$$\sigma \in \{0.01, 0.1, 1, 10\}.$$

Compare per-tensor scaling and per-block scaling. Measure RMS relative error.

Expected result:

$$\text{error}_{\text{block scale}} < \text{error}_{\text{tensor scale}}$$

when blocks have different dynamic ranges.

Backward Determinism Experiment

Implement two versions of a gradient accumulation kernel:

- one using atomic adds,
- one using deterministic reduction order.

Run the same input multiple times and compare bitwise equality. This illustrates why one-kernel backward can be nondeterministic and why two-kernel backward is attractive.

Nsight Compute Investigation

Profile an attention kernel and inspect:

- tensor-core utilization,
- shared-memory throughput,
- global-memory throughput,
- register spills,
- occupancy,
- barrier stalls,
- instruction mix for exponentials and FP32 operations.

The goal is to classify the kernel as:

MMA-bound, softmax-bound, memory-bound, synchronization-bound.

Final Mental Model

cuDNN attention kernels are built by treating attention as a carefully scheduled pipeline, not as a pair of matrix multiplications. The forward pass has:

$$QK^T \rightarrow \text{softmax} \rightarrow PV.$$

The backward pass has multiple BMMs, softmax-derived math, and gradient accumulation. On modern GPUs, the kernel designer must arrange these pieces so that tensor cores stay busy while global-memory loads, exponentials, reductions, quantization, and synchronization are hidden.

BF16 and FP16 attention are already pipeline-design problems. FP8 makes the problem sharper because MMA throughput increases, causing softmax and dsoftmax to become more exposed. Classical FP8 can remain efficient by using per-tensor scales, power-of-two scale selection, historical amax for dS , and clever fusion of the probability scale into the base-2 exponent.

MXFP8 improves numerical quality by using block-level scales, but the cost is a more complex hardware pipeline. Scales must be loaded, copied into tensor memory, aligned with the MMA contracting dimension, and protected from overwriting by accumulators. Intermediate tensors such as dS may require online block amax computation, which adds CUDA-math work that cannot be fully hidden.

The core lesson is:

Low precision only wins when the entire attention pipeline is re-designed around the new balance of hardware costs. Faster MMAs expose everything else. The art of cuDNN FP8 and MXFP8 attention is to make the non-MMA work disappear behind the tensor-core schedule while preserving enough numerical fidelity for training and inference.